

LegalEase

AI Document Plan Generator

Complete project report and implementation reference

Project stack	Streamlit • FastAPI • Google Gemini • python-docx • fpdf2
Core workflow	User input → FastAPI → Gemini → editable preview → TXT/DOCX/PDF
Development mode	Gemini API mode + local mock mode for testing
Source basis	Supplied LegalEase project documentation and completed project code

This report documents the implemented application and the setup needed to run it locally in VS Code. The supplied specification calls for Streamlit, FastAPI, Gemini-based generation, editable previews, and TXT/DOCX/PDF export.

1. LegalEase — AI Document Plan Generator

LegalEase is an AI-assisted legal document drafting application designed to simplify the creation of structured documents such as employment contracts, lease agreements, NDAs, service agreements, freelance contracts, and general agreements.

The user supplies the document type, parties, terms and conditions, effective date, optional jurisdiction, and language. The backend validates these inputs, sends a structured prompt to Gemini when API mode is enabled, and returns generated text for review and editing.

Primary objective

Provide a simple workflow for generating a professional-looking draft, reviewing/editing it, and exporting it in common document formats without requiring users to manually assemble the document structure.

Document examples from the project specification

- Employment Contract
- Lease Agreement
- Non-Disclosure Agreement (NDA)
- Freelance Work Contract
- Service Agreement
- General Agreement

2. Architecture

The application follows a three-layer architecture: Streamlit frontend, FastAPI backend, and Gemini AI core. Formatting/export utilities remain separated from the API and AI modules.

Layer	Component	Responsibility
Frontend	Streamlit (app.py)	Collect inputs, call API, show editable preview, provide downloads.
Backend	FastAPI (backend/main.py, routes.py)	Health endpoints, validation, generation API, error handling.
AI Core	GeminiDocumentGenerator	Build prompt, call Gemini, or use mock generation when enabled.
Export	utils/document_export.py	Generate TXT, DOCX and PDF output with formatting/footer.
Utilities	utils/sanitize.py	Normalize typography/control characters and make PDF-safe text.

Request flow

1. User enters document requirements in Streamlit.
2. Streamlit sends POST /generate to FastAPI.
3. Pydantic validates the request fields and size limits.
4. GeminiDocumentGenerator creates the draft (or mock content in test mode).
5. FastAPI returns the generated text and model/mock status.
6. Streamlit places the text in an editable text area.
7. The user previews and exports the final edited draft as TXT, DOCX, or PDF.

3. Features

AI legal-document drafting — Generates structured draft documents from user-supplied facts and terms.

Editable preview — Generated text can be modified directly before export.

Multi-format export — TXT, DOCX and PDF download options are provided.

Document branding — DOCX/PDF exporters support an optional logo and include a LegalEase footer.

Structured clauses — Semicolon-separated terms are converted into clauses in the generated/mock draft.

Input validation — FastAPI/Pydantic enforces required fields and reasonable input-length limits.

Mock mode — The application can run without a Gemini key for local UI and integration testing.

Language field — The UI provides English, Tamil, Hindi and Other as language choices.

Jurisdiction field — An optional jurisdiction can be supplied to the generation prompt.

API documentation — FastAPI automatically exposes interactive OpenAPI documentation at /docs.

4. Safety Note

LegalEase is an AI-assisted drafting tool, not a substitute for a qualified legal professional. The generated text should be treated as a draft and reviewed before it is signed, submitted, or relied upon.

- The Gemini prompt instructs the model to use only user-supplied facts and not invent names, dates, amounts, addresses, statutes, citations, or jurisdiction-specific requirements.
- When important information is missing, the implementation asks the model to use [TO BE COMPLETED] rather than guessing.
- The generated document explicitly avoids claiming that it is legally sufficient or attorney-reviewed.
- Users should verify all parties, dates, obligations, amounts, governing law, signatures, and other legal requirements before use.
- API keys belong in .env and should not be committed to source control.

5. Quick Start — Windows / VS Code

Step 1 — Open the project

Extract LegalEase.zip and open the extracted LegalEase folder in Visual Studio Code.

Step 2 — Create a virtual environment

```
py -3.11 -m venv .env  
.\.env\Scripts\Activate.ps1
```

Step 3 — Install dependencies

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

Step 4 — Configure environment

Copy .env.example to .env

GEMINI_API_KEY=your_google_ai_studio_key_here

```
GEMINI_MODEL=gemini-3.8-flash
BACKEND_URL=http://127.0.0.1:8000
LEGAL_EASE MOCK=false
```

Step 5 — Start FastAPI

```
uvicorn backend.main:app --reload --port 8000
```

Backend: <http://127.0.0.1:8000> | Swagger UI: <http://127.0.0.1:8000/docs>

Step 6 — Start Streamlit in a second VS Code terminal

```
streamlit run app.py
```

Open the Streamlit URL shown in the terminal, normally <http://localhost:8501>.

Test without a Gemini key

Set `LEGAL_EASE MOCK=true` in `.env`. This exercises the complete frontend → backend → generation → export workflow without calling Gemini.

6. Test

The project includes automated tests for the FastAPI health endpoint, mock document generation, typography sanitization, and PDF-safe text conversion.

```
pytest -q
```

Validated project result: 4 tests passed.

Test	Purpose
test_health	Confirms GET /health returns HTTP 200 and healthy status.
test_generate_mock	Confirms POST /generate produces an NDA draft in mock mode.
test_sanitize_replaces_typography	Confirms curly quotes, em dash and ellipsis are normalized.
test_pdf_safe_is_latin1	Confirms PDF-safe text is returned as a string suitable for the PDF exporter.

7. REST API

GET /

Returns basic service metadata and points to the API documentation.

```
{"name": "LegalEase API", "status": "ok", "docs": "/docs"}
```

GET /health

Health-check endpoint used by tests and deployment checks.

```
{"status": "healthy"}
```

POST /generate

Accepts a validated DocumentRequest and returns generated document text.

```
{
  "document_type": "NDA",
  "parties": "Alice (Disclosing Party), Bob (Receiving Party)",
  "terms": "Keep information confidential; return information on termination",
  "dates": "October 2, 2026",
  "jurisdiction": "",
  "language": "English"
}
```

Response shape

```
{
  "document": "...generated text...",
  "model": "gemini-3.8-flash",
  "mock": false
}
```

Interactive API testing is available through FastAPI Swagger UI at /docs after the backend is running.

8. Model Configuration

The model configuration is environment-driven so the application code does not need to change when the configured Gemini model changes.

Variable	Default / Example	Purpose
GEMINI_API_KEY	your_google_ai_studio_key_here	API credential used by the Google GenAI client.
GEMINI_MODEL	gemini-3.8-flash	Gemini model identifier used for document generation.
LEGAL_EASE MOCK	false	When true, uses deterministic local mock generation instead of Gemini.
BACKEND_URL	http://127.0.0.1:8000	Streamlit target for the FastAPI backend.

Generation design

- The prompt identifies the application as LegalEase, a legal-document drafting assistant.
- Only supplied facts should be used; missing critical information should become [TO BE COMPLETED].
- The output is plain text rather than Markdown fences.
- The output should include a title, parties, effective date, document-appropriate sections, clauses and signature blocks.
- The prompt explicitly avoids claiming legal sufficiency or attorney review.

Project Structure

```
LegalEase/
├── app.py
├── .env.example
├── .gitignore
├── requirements.txt
├── README.md
├── ai_core/
│   ├── __init__.py
│   └── gemini_generator.py
├── backend/
│   ├── __init__.py
│   ├── main.py
│   └── routes.py
├── utils/
│   ├── __init__.py
│   ├── document_export.py
│   └── sanitize.py
├── tests/
│   ├── test_api.py
│   └── test_sanitize.py
└── assets/
    └── .gitkeep
```

The ZIP also contains Python cache files generated during validation; they are not required for a fresh installation.

Code Appendix — Core Implementation

The following appendix reproduces the working source files from the completed project so the PDF can serve as a self-contained implementation reference. The ZIP remains the preferred form for actually running the application.

app.py

```
import os
import html
import requests
import streamlit as st
from dotenv import load_dotenv
from utils.document_export import format_txt, format_docx, format_pdf

load_dotenv()
st.set_page_config(page_title="LegalEase", page_icon="⚖️", layout="wide")

BACKEND_URL = os.getenv("BACKEND_URL", "http://127.0.0.1:8000").rstrip("/")

st.markdown("""
<style>
.main-title { text-align:center; font-size:2.5rem; font-weight:700; margin-bottom:0; }
.subtitle { text-align:center; color:#8b95a7; margin-bottom:2rem; }
.preview { background:#f3f4f6; border-radius:12px; padding:24px;
max-height:650px; overflow:auto; white-space:pre-wrap; font-family:Georgia,serif; }
.disclaimer { font-size:.85rem; color:#6b7280; }
</style>
""", unsafe_allow_html=True)

st.markdown('<div class="main-title">⚖️ LegalEase</div>', unsafe_allow_html=True)
st.markdown('<div class="subtitle">AI-powered legal document drafting and export</div>', unsafe_allow_html=True)

with st.sidebar:
    st.header("Document Settings")
    document_type = st.selectbox(
        "Document type",
        ["Employment Contract", "Lease Agreement", "Non-Disclosure Agreement (NDA)",
        "Freelance Work Contract", "Service Agreement", "General Agreement", "Other"]
    )
    if document_type == "Other":
        document_type = st.text_input("Specify document type")
    language = st.selectbox("Language", ["English", "Tamil", "Hindi", "Other"])
    jurisdiction = st.text_input("Jurisdiction (optional)", placeholder="e.g. Tamil Nadu, India")
    dates = st.text_input("Effective date", placeholder="10/04/2026")

st.subheader("1. Parties")
parties = st.text_area(
    "Names and roles",
    placeholder="Jane Doe (Service Provider), TechNova Inc. (Client)",
    height=100,
)

st.subheader("2. Terms & Conditions")
terms = st.text_area(
    "Use semicolons to separate terms",
    placeholder="Payment within 30 days; confidentiality must be maintained; either party may terminate with 15 days notice",
    height=150,
)

if st.button("⏏ Generate Document", type="primary", use_container_width=True):
    if not all([document_type.strip(), parties.strip(), terms.strip(), dates.strip()]):
        st.error("Please complete document type, parties, terms, and effective date.")
    else:
        payload = {
            "document_type": document_type,
            "parties": parties,
            "terms": terms,
            "dates": dates,
            "jurisdiction": jurisdiction,
            "language": language,
        }
        try:
            with st.spinner("Generating your document..."):
                response = requests.post(f"{BACKEND_URL}/generate", json=payload, timeout=180)
                if response.ok:
                    data = response.json()
                    st.session_state.document = data["document"]
                    st.session_state.doc_type = document_type
                    st.success("Document generated.")
                    if data.get("mock"):
                        st.info("Running in mock mode because GEMINI_API_KEY is not configured.")
                else:
                    detail = response.json().get("detail", response.text)
                    st.error(f"Backend error: {detail}")
            except requests.RequestException as exc:
                st.error(f"Could not reach FastAPI at {BACKEND_URL}: {exc}")

if "document" in st.session_state:
    st.divider()
    st.subheader("3. Preview & Edit")
    edited = st.text_area(
        "Editable document",
        value=st.session_state.document,
        height=500,
        label_visibility="collapsed",
    )
```

```

)
st.session_state.document = edited

st.markdown("***Preview**")
st.markdown(
    f'<div class="preview">{html.escape(edited)}</div>',
    unsafe_allow_html=True,
)

st.subheader("4. Download")
c1, c2, c3 = st.columns(3)
filename = "".join(c if c.isalnum() or c in "-_" else "_" for c in st.session_state.doc_type.lower()).strip("_") or "legalease_document"

with c1:
    st.download_button("TXT", format_txt(edited), f"{filename}.txt", "text/plain", use_container_width=True)
with c2:
    st.download_button("DOCX", format_docx(edited, st.session_state.doc_type), f"{filename}.docx",
        "application/vnd.openxmlformats-officedocument.wordprocessingml.document", use_container_width=True)
with c3:
    st.download_button("PDF", format_pdf(edited, st.session_state.doc_type), f"{filename}.pdf",
        "application/pdf", use_container_width=True)

st.markdown('<p class="disclaimer">LegalEase generates drafts for review and does not replace advice from a qualified legal professional.</p>', unsafe_allow_html=True)

```

backend/main.py

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.routes import router

app = FastAPI(
    title="LegalEase API",
    version="1.0.0",
    description="AI-powered legal document generation API.",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(router)

@app.get("/", tags=["health"])
def root():
    return {"name": "LegalEase API", "status": "ok", "docs": "/docs"}

@app.get("/health", tags=["health"])
def health():
    return {"status": "healthy"}

```

backend/routes.py

```

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str = Field(..., min_length=2, max_length=120)
    parties: str = Field(..., min_length=2, max_length=5000)
    terms: str = Field(..., min_length=2, max_length=10000)
    dates: str = Field(..., min_length=2, max_length=300)
    jurisdiction: str = Field(default="", max_length=300)
    language: str = Field(default="English", max_length=80)

@router.post("/generate", tags=["documents"])
def generate_document(request: DocumentRequest):
    try:
        text = generator.generate_document(
            document_type=request.document_type,
            parties=request.parties,
            terms=request.terms,
            dates=request.dates,
            jurisdiction=request.jurisdiction,
            language=request.language,
        )
        return {"document": text, "model": generator.model_name, "mock": generator.mock_mode}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=f"Document generation failed: {exc}") from exc

```

ai_core/gemini_generator.py

```

import os
from dotenv import load_dotenv

load_dotenv()

class GeminiDocumentGenerator:
    def __init__(self):
        self.model_name = os.getenv("GEMINI_MODEL", "gemini-3.8-flash")
        self.api_key = os.getenv("GEMINI_API_KEY", "").strip()
        self.mock_mode = os.getenv("LEGAL_EASE MOCK", "false").lower() == "true" or not self.api_key

```

```

        self.client = None
        if not self.mock_mode:
            from google import genai
            self.client = genai.Client(api_key=self.api_key)

    def generate_document(self, document_type, parties, terms, dates, jurisdiction="", language="English"):
        if self.mock_mode:
            return self._mock_document(document_type, parties, terms, dates, jurisdiction, language)

        prompt = f"""
You are LegalEase, a legal-document drafting assistant.

Draft a professional, structured {document_type}.
Use ONLY the user-supplied facts below. Do not invent names, dates, amounts, addresses,
obligations, statutes, citations, or jurisdiction-specific legal requirements.
If important information is missing, insert [TO BE COMPLETED] rather than guessing.

USER INPUT
Document type: {document_type}
Parties: {parties}
Terms/conditions: {terms}
Effective date: {dates}
Jurisdiction: {jurisdiction or "[NOT SPECIFIED]"}
Language: {language}

OUTPUT REQUIREMENTS
- Return plain text only; no Markdown fences.
- Start with a clear document title.
- Include sections appropriate to the document type.
- Include a "PARTIES" section and an "EFFECTIVE DATE" section.
- Convert each semicolon-separated term into a clear clause where appropriate.
- Include signature blocks for the relevant parties.
- Use neutral, professional legal drafting.
- Do not claim the document is legally sufficient or attorney-reviewed.
"""

        response = self.client.models.generate_content(
            model=self.model_name,
            contents=prompt,
        )
        text = getattr(response, "text", None)
        if not text:
            raise RuntimeError("Gemini returned an empty response.")
        return text.strip()

    @staticmethod
    def _mock_document(document_type, parties, terms, dates, jurisdiction, language):
        bullets = [t.strip() for t in terms.replace("\n", ";").split(";") if t.strip()]
        clauses = "\n".join(f"{i}. {term}" for i, term in enumerate(bullets, 1))
        return f"""{document_type.upper()}

PARTIES
{parties}

EFFECTIVE DATE
{dates}

JURISDICTION
{jurisdiction or "[TO BE COMPLETED]"}

RECITALS
The parties identified above intend to enter into this {document_type} on the effective date stated above.

TERMS AND CONDITIONS
{clauses or "1. [TO BE COMPLETED]"}

GENERAL
This document is a generated draft for review. Complete all placeholders and have the document reviewed by an appropriately qualified legal professional before use.

SIGNATURES

Party 1: _____
Name: _____
Date: _____

Party 2: _____
Name: _____
Date: _____
"""

```

utils/sanitize.py

```

import re

def sanitize_text(text: str) -> str:
    if not text:
        return ""
    replacements = {
        "\u2018": "'", "\u2019": "'", "\u201c": '"', "\u201d": '"',
        "\u2013": "-", "\u2014": "-", "\u2026": "...", "\u20a0": " ",
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    text = re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f]", "", text)
    return text.strip()

def pdf_safe(text: str) -> str:
    return sanitize_text(text).encode("latin-1", "replace").decode("latin-1")

```

utils/document_export.py

```

from io import BytesIO
from pathlib import Path
from typing import Optional
from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.shared import Inches, Pt
from fpdf import FPDF
from utils.sanitize import sanitize_text, pdf_safe

BASE_DIR = Path(__file__).resolve().parents[1]
LOGO_PATH = BASE_DIR / "assets" / "logo.png"

def format_txt(text: str) -> bytes:
    return sanitize_text(text).encode("utf-8")

def _add_logo_docx(doc: Document):
    if LOGO_PATH.exists():
        p = doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        p.add_run().add_picture(str(LOGO_PATH), width=Inches(1.2))

def format_docx(text: str, doc_type: str) -> bytes:
    doc = Document()
    sec = doc.sections[0]
    sec.top_margin = Inches(0.75)
    sec.bottom_margin = Inches(0.75)
    _add_logo_docx(doc)

    title = doc.add_paragraph()
    title.alignment = WD_ALIGN_PARAGRAPH.CENTER
    run = title.add_run(doc_type.upper())
    run.bold = True
    run.font.name = "Times New Roman"
    run.font.size = Pt(16)

    for raw in sanitize_text(text).splitlines():
        line = raw.strip()
        if not line:
            doc.add_paragraph()
            continue
        p = doc.add_paragraph()
        p.paragraph_format.space_after = Pt(6)
        r = p.add_run(line)
        r.font.name = "Times New Roman"
        r.font.size = Pt(11)
        if (line.isupper() and len(line) < 80) or line.endswith(":"):
            r.bold = True

    footer = sec.footer.paragraphs[0]
    footer.alignment = WD_ALIGN_PARAGRAPH.CENTER
    fr = footer.add_run("LegalEase – AI-generated draft. Review before use.")
    fr.font.size = Pt(8)

    out = BytesIO()
    doc.save(out)
    return out.getvalue()

class LegalEasePDF(FPDF):
    def __init__(self, doc_type):
        super().__init__()
        self.doc_type = doc_type
        self.set_auto_page_break(True, margin=18)

    def header(self):
        if LOGO_PATH.exists():
            try:
                self.image(str(LOGO_PATH), x=95, y=8, w=20)
                self.ln(20)
            except Exception:
                pass
        self.set_font("Times", "B", 10)
        self.cell(0, 8, pdf_safe(self.doc_type), align="C")
        self.ln(8)

    def footer(self):
        self.set_y(-15)
        self.set_font("Times", "", 8)
        self.cell(0, 8, "LegalEase - AI-generated draft. Review before use.", align="C")

def format_pdf(text: str, doc_type: str) -> bytes:
    pdf = LegalEasePDF(doc_type)
    pdf.add_page()
    for raw in sanitize_text(text).splitlines():
        line = raw.strip()
        if not line:
            pdf.ln(4)
            continue
        is_heading = (line.isupper() and len(line) < 80) or line.endswith(":")
        pdf.set_font("Times", "B" if is_heading else "", 11)
        pdf.multi_cell(0, 6, pdf_safe(line))
        pdf.ln(1)
    return bytes(pdf.output())

```

tests/test_api.py

```

from fastapi.testclient import TestClient
from backend.main import app

client = TestClient(app)

def test_health():

```

```

response = client.get("/health")
assert response.status_code == 200
assert response.json()["status"] == "healthy"

def test_generate_mock(monkeypatch):
    monkeypatch.setenv("LEGAL_EASE MOCK", "true")
    response = client.post("/generate", json={
        "document_type": "NDA",
        "parties": "Alice (Disclosing Party), Bob (Receiving Party)",
        "terms": "Keep information confidential; return information on termination",
        "dates": "October 2, 2026",
    })
    assert response.status_code == 200
    assert "NDA" in response.json()["document"]

```

tests/test_sanitize.py

```

from utils.sanitize import sanitize_text, pdf_safe

def test_sanitize_replaces_typography():
    assert sanitize_text("'Hello" - world...") == "'Hello" - world...'

def test_pdf_safe_is_latin1():
    assert isinstance(pdf_safe("café"), str)

```

requirements.txt

```

fastapi>=0.115,<1
uvicorn[standard]>=0.34,<1
streamlit>=1.42,<2
google-generativeai>=1.30,<2
python-dotenv>=1.0,<2
requests>=2.32,<3
python-docx>=1.1,<2
fpdf2>=2.8,<3
Pillow>=10,<13
pytest>=8,<9
httpx>=0.27,<1

```

.env.example

```

# Copy this file to .env
GEMINI_API_KEY=your_google_ai_studio_key_here
GEMINI_MODEL=gemini-3.8-flash
BACKEND_URL=http://127.0.0.1:8000

# Set true to run without an API key for local UI/testing.
LEGAL_EASE MOCK=false

```

End of report. For the runnable application, use the accompanying LegalEase.zip package.